CSC 34200

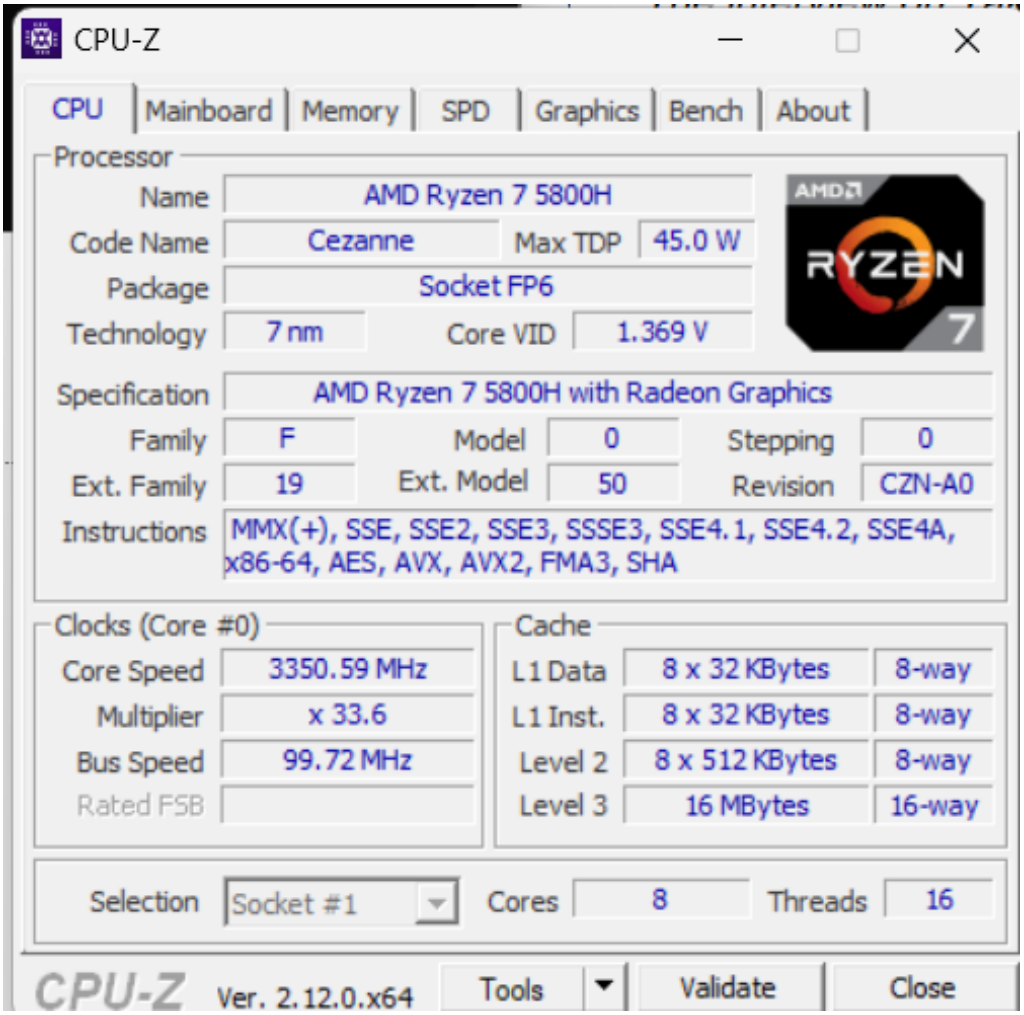
Final Take Home Exam Dot Product

Allan Yunayev

12/15/2024

Objective: The objective of this final take home test is to optimize compiler generated code to compute dot product using vector instructions. We are to perform different tasks in this take home exam.

Task 1: Use CPUID instruction to determine your processor vector processing capabilities



Task 2: Write C++ function to compute dot product in Visual Studio environment. Place the function in a separate file from main() that calls this function. Vector sizes should be powers of 2 (e.g. 16, 32, 64, …..512, ..216 etc.). Disable Automatic Parallelization, /Qpar, and Automatic Vectorization, /arch. Use high resolution timer to measure execution time.

Enable String Pooling	
Enable Minimal Rebuild	No (/Gm-)
Enable C++ Exceptions	Yes (/EHsc)
Smaller Type Check	No
Basic Runtime Checks	**Default**
Runtime Library	Multi-threaded Debug DLL (/MDd)
Struct Member Alignment	Default
Security Check	Enable Security Check (/GS)
Control Flow Guard	
Enable Function-Level Linking	
Enable Parallel Code Generation	**No (/Qpar-)**
Enable Enhanced Instruction Set	**No Enhanced Instructions (/arch:IA32)**
Floating Point Model	Precise (/fp:precise)
Enable Floating Point Exceptions	
Create Hotpatchable Image	
Spectre Mitigation	Disabled
Enable Intel JCC Erratum Mitigation	No
Enable EH Continuation Metadata	
Enable Signed Returns	

we must turn off these setting in Visual Studio

```h
#pragma once
#ifndef DotProduct_H
#define DotProduct_H

// Function declaration
int DotProduct(int A[], int B[], int C, int size);

#endif // DOTPRODUCT_H
```

DotProduct.h

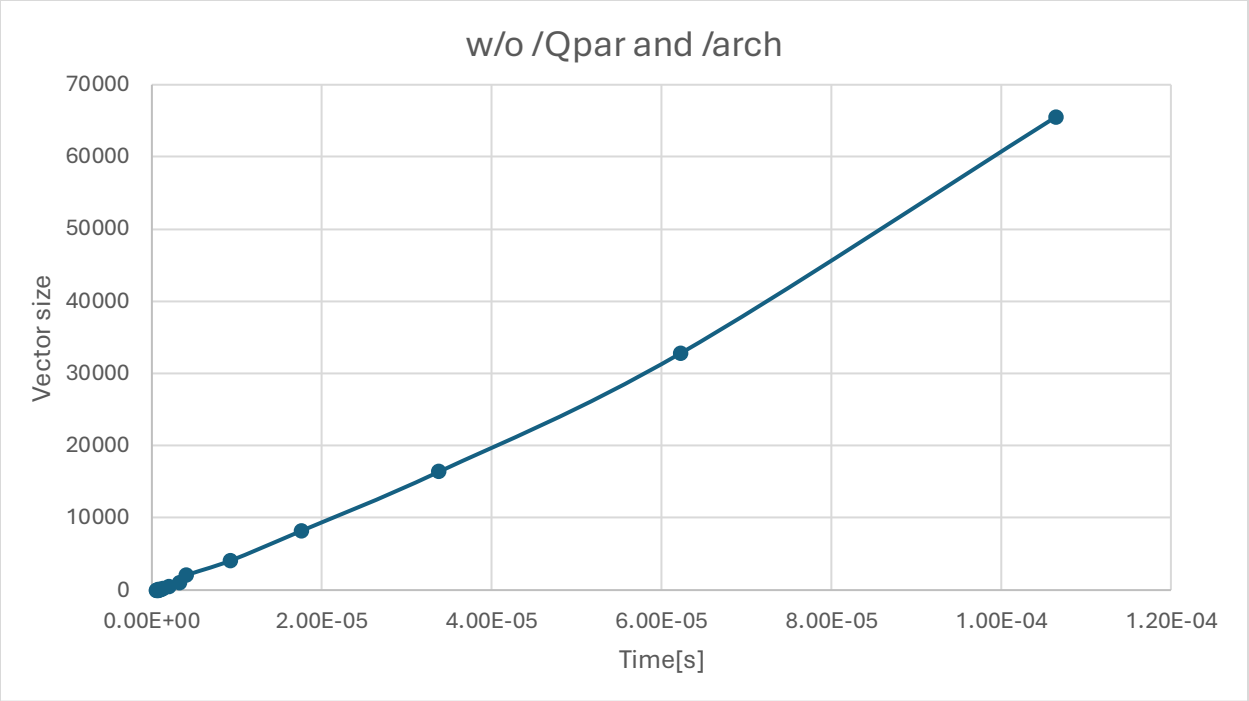
```cpp
#include <iostream>
#include "DotProduct.h"
using namespace std;
int DotProduct(int A[] ,int B[], int C, int size){
    for(int i =0;i<size;i++){
        C = C +A[i]*B[i];
    }
    return C;
}

```

DotProduct.cpp that is in separate file from main

```cpp
#include <iostream>
#include <windows.h>
#include "DotProduct.h"
using namespace std;

double PCFreq = 0.0;
__int64 CounterStart = 0;

void StartCounter()
{
    LARGE_INTEGER li;
    if (!QueryPerformanceFrequency(&li))
        cout << "QueryPerformanceFrequency failed!\n";
    PCFreq = double(li.QuadPart) / 1.0;
    QueryPerformanceCounter(&li);
    CounterStart = li.QuadPart;
}

double GetCounter()
{
    LARGE_INTEGER li;
    QueryPerformanceCounter(&li);
    return double(li.QuadPart - CounterStart) / PCFreq;
}

int main()
{
    int A[65536];
    int B[65536];
    for (int i = 0; i < 65536; i++) {
        A[i] = 1;
        B[i] = 2;
    }
    int C=0;
    StartCounter();
    C=DotProduct(A,B,C,65536);
    cout<<GetCounter()<<" s"<<endl;
    return 0;
}
```

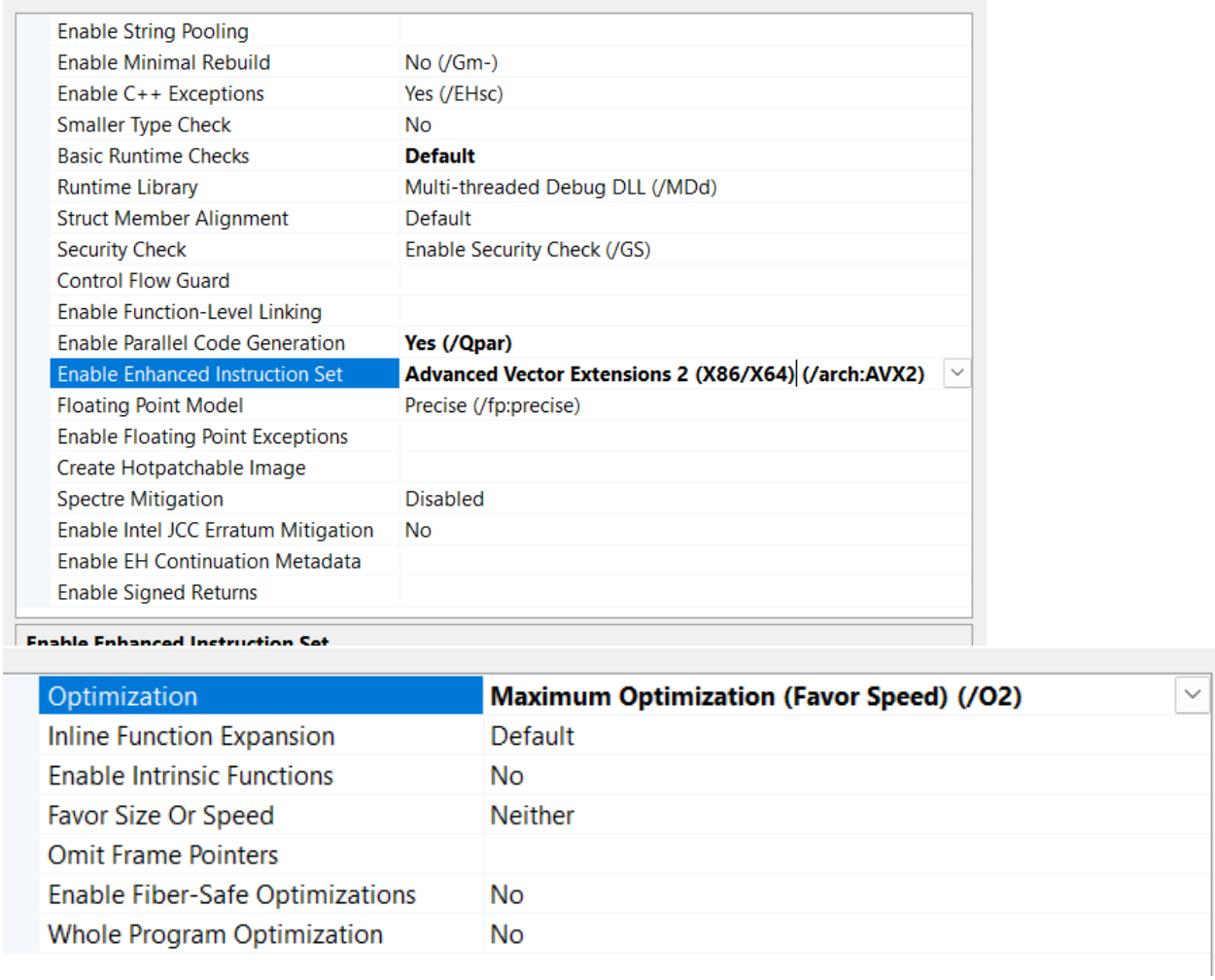
DotMain.cpp

w/o /Qpar and /arch	
Time[s]	Dot Product Size
6.50E-07	1
5.33E-07	2
5.30E-07	4
7.60E-07	8
8.22E-07	16
7.56E-07	32
1.16E-06	64
9.58E-07	128
1.29E-06	256
1.99E-06	512
3.29E-06	1024
4.10E-06	2048
9.29E-06	4096
1.76E-05	8192
3.38E-05	16384
6.22E-05	32768
1.06E-04	65536



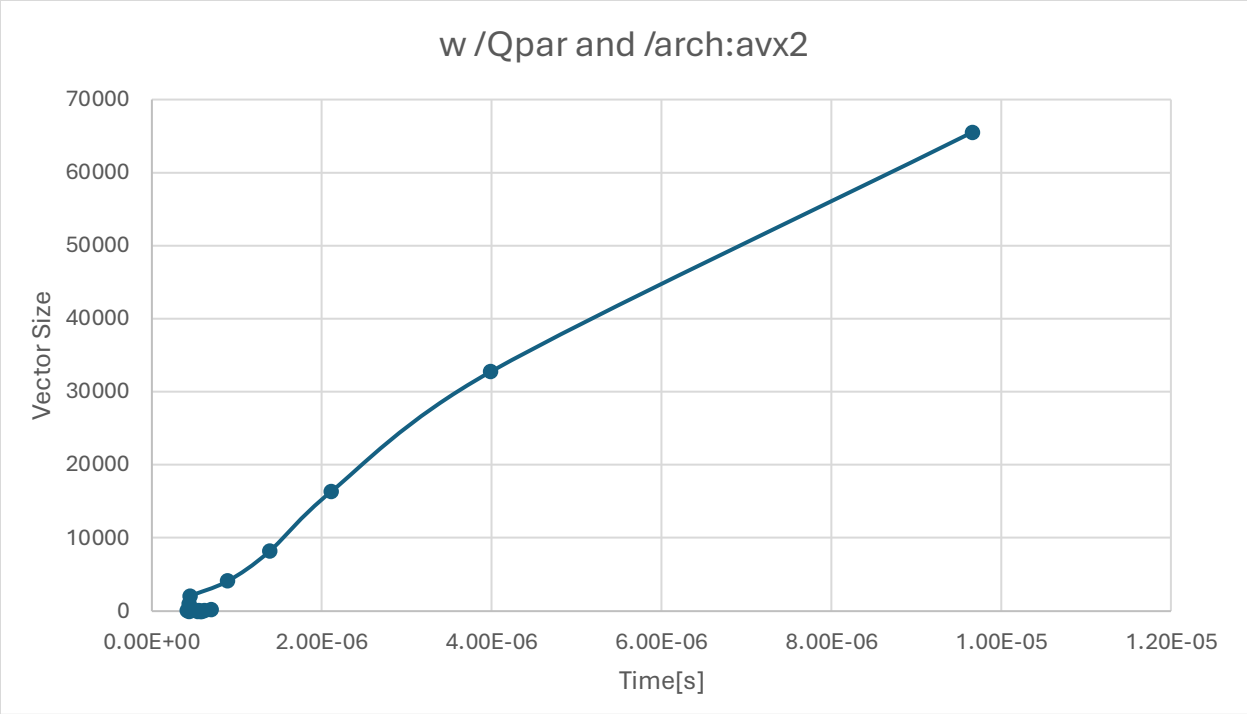
Graph of Dot main without /Qpar /arch

Task 3: Compile code in tak 2. Enable Automatic Parallelization, /Qpar, and Automatic Vectorization, /arch. Use high resolution timer chrono library function to measure execution time. Plot graph: time versus vector size. Inspect compiler generated assembly code. Observe if compiler vectorized code for very large vector sizes. Try to optimize compiler generated code. Based on compiler generated assembly code (or your optimized code) create an assembly code for dot product computation function



You have to turn on optimization for AVX2 to work

w /Qpar and /arch:avx2	
Time[s]	Dot Product Size
4.40E-07	1
5.83E-07	2
5.75E-07	4
5.40E-07	8
4.17E-07	16
6.14E-07	32
4.67E-07	64
5.56E-07	128
7.00E-07	256
4.46E-07	512
4.38E-07	1024
4.55E-07	2048
8.91E-07	4096
1.39E-06	8192
2.11E-06	16384
3.99E-06	32768
9.66E-06	65536



Graph w /Qpar and /AVX2

```
_TEXT	SEGMENT
i$1 = 0
A$ = 96
B$ = 104
C$ = 112
size$ = 120
?DotProduct@@YAHQEAH0HH@Z PROC                          ; DotProduct, COMDAT
; 7   : int DotProduct(int A[] ,int B[], int C, int size){
$LN6:
	mov	DWORD PTR [rsp+32], r9d
	mov	DWORD PTR [rsp+24], r8d
	mov	QWORD PTR [rsp+16], rdx
	mov	QWORD PTR [rsp+8], rcx
	push	rbp
	sub	rsp, 112                                ; 00000070H
	lea	rbp, QWORD PTR [rsp+32]
	lea	rcx, OFFSET FLAT:__4BE2BEFE_DotProduct@cpp
	call	__CheckForDebuggerJustMyCode

; 8   :   for(int i =0;i<size;i++){
	mov	DWORD PTR i$1[rbp], 0  ; change to mov eax, 0 REMOVED
	; NEW ADDED ASM  added all initalization
	; mov eax, 0 ; new i instead of prt of I
	;mov	rdx, QWORD PTR A$[rbp]
	;mov	r8, QWORD PTR B$[rbp]
	;mov	ecx, DWORD PTR C$[rbp]
	;mov	edx, DWORD PTR size$[rbp]
	jmp	SHORT $LN4@DotProduct
$LN2@DotProduct:
	mov	eax, DWORD PTR i$1[rbp] ; remove this so i is not in stack REMOVE
	inc	eax ; increment I by 1
	mov	DWORD PTR i$1[rbp], eax ; again remove we this REMOVE
$LN4@DotProduct:
mov	eax, DWORD PTR size$[rbp] ; replace eax with edx and add to initlztion

cmp	DWORD PTR i$1[rbp], eax	; cmp eax, edx REMOVE
    ;cmp eax,edx ; compare i<size
    jge	SHORT $LN3@DotProduct ; jump to end when done

; 9   :	C = C +A[i]*B[i];

movsxd	rax, DWORD PTR i$1[rbp] ;replace Dword ptr i$1 with eax REMOVE
movsxd	rcx, DWORD PTR i$1[rbp] ;remove as it is already rax  REMOVE
    ;movsxd rax, eax  ; create sign-extension of I to rax
mov	rdx, QWORD PTR A$[rbp] ;REMOVE
mov	r8, QWORD PTR B$[rbp]; REMOVE
mov	eax, DWORD PTR [rdx+rax*4] ;REMOVE change eax to edi
    ;mov edi, DWORD PTR [rdx+rax*4]  ; move A[i] to esi
imul	eax, DWORD PTR [r8+rcx*4] ; REMOVE change eax to edi  and change it too rax*4
    ;imul edi, DWORD PTR [r8+rax*4]  ; multiply B[i] with A[i] that in edi
mov	ecx, DWORD PTR C$[rbp] ;REMOVE move to initalization
; take all rdx, r8, ecx into the initialize part
add	ecx, eax ;REMOVE add ecx,edi
    ;add ecx,edi ; add edi to C
mov	eax, ecx ;REMOVE mov edi,ecx
    ;mov edi,ecx  move c to edi
mov	DWORD PTR C$[rbp], eax ;REMOVE  eax, to edi
    ;mov	DWORD PTR C$[rbp], edi ; move edi to prt C
; 10  :   }

    jmp	SHORT $LN2@DotProduct
$LN3@DotProduct:

; 11  :   return C;

    mov	eax, DWORD PTR C$[rbp]

; 12  : }

    lea	rsp, QWORD PTR [rbp+80]
    pop	rbp
    ret	0
```

Red is the optimization set up all the initializations out of the loop and turn i to eax and not let it go into stack.

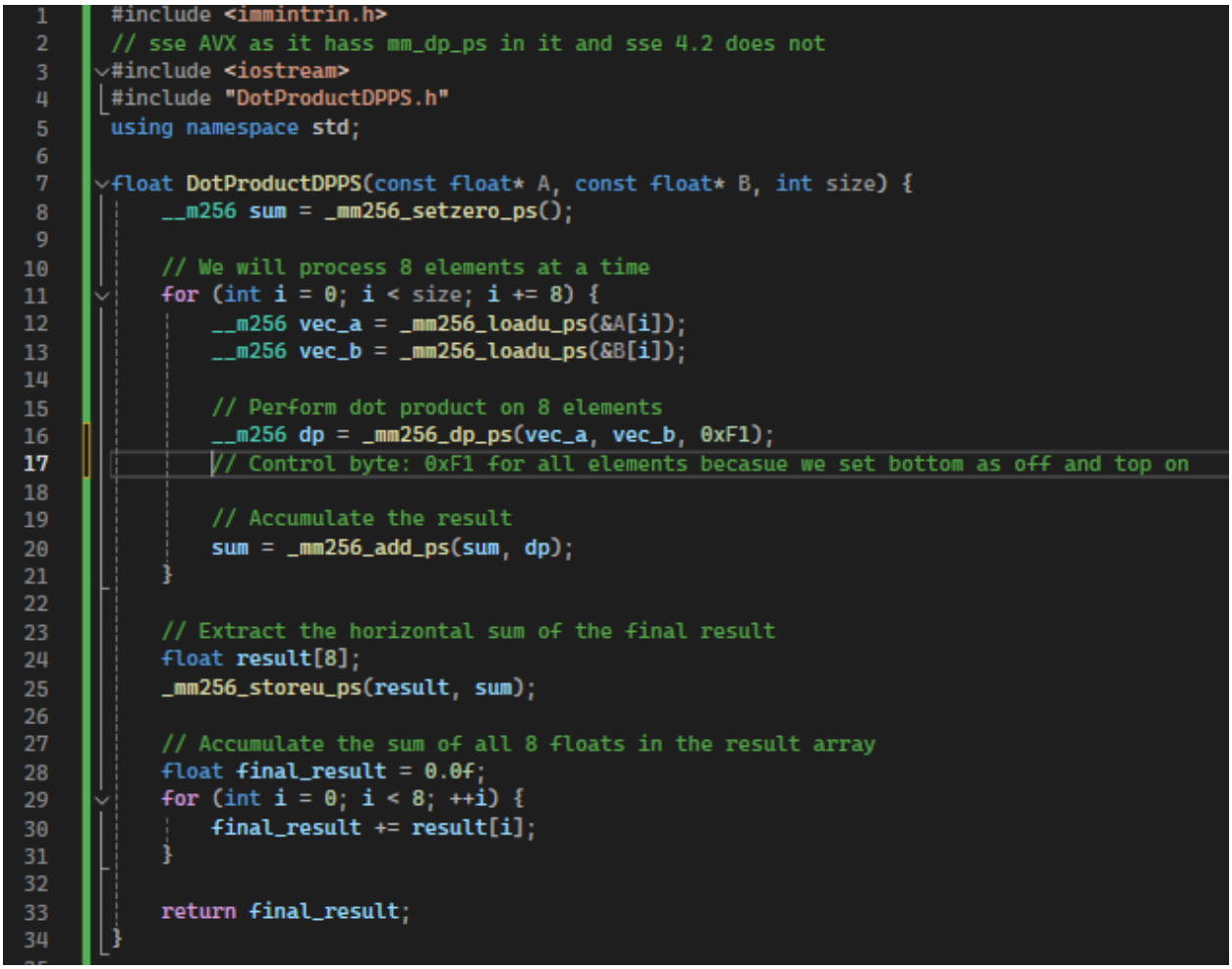
Task 4: To optimize the code further, please try to use vector instruction DPPS to compute dot product. Use high resolution timer to measure execution time. Plot graph: time versus vector size

```cpp
#include <immintrin.h>
// AVX as it hass mm_dp_ps in it and AVX does not
#include <iostream>
#include <windows.h>
#include "DotProductDPPS.h"
using namespace std;

double PCFreq = 0.0;
__int64 CounterStart = 0;

void StartCounter()
{
    LARGE_INTEGER li;
    if (!QueryPerformanceFrequency(&li))
        cout << "QueryPerformanceFrequency failed!\n";
    PCFreq = double(li.QuadPart);
    QueryPerformanceCounter(&li);
    CounterStart = li.QuadPart;
}

double GetCounter()
{
    LARGE_INTEGER li;
    QueryPerformanceCounter(&li);
    return double(li.QuadPart - CounterStart) / PCFreq;
}
int main()
{
        int size = 65536;
        float A[65536], B[65536];

        // Initialize arrays
        for (int i = 0; i < size;++ i) {
            A[i] = 1.0;
            B[i] = 2.0;
        }
        StartCounter();
        float result = DotProductDPPS(A, B, size);
        cout << "Dot Product Time: " << GetCounter() << " S"<<endl;

        return 0;
}
```

DotDPPSmain.cpp

```cpp
#pragma once
#ifndef DotProductDPPS_H
#define DotProductDPPS_H

// Function declaration
float DotProductDPPS(const float* A,const float* B, int size);

#endif // DOTPRODUCTDPPS_H
```

DotProductDPPS.h

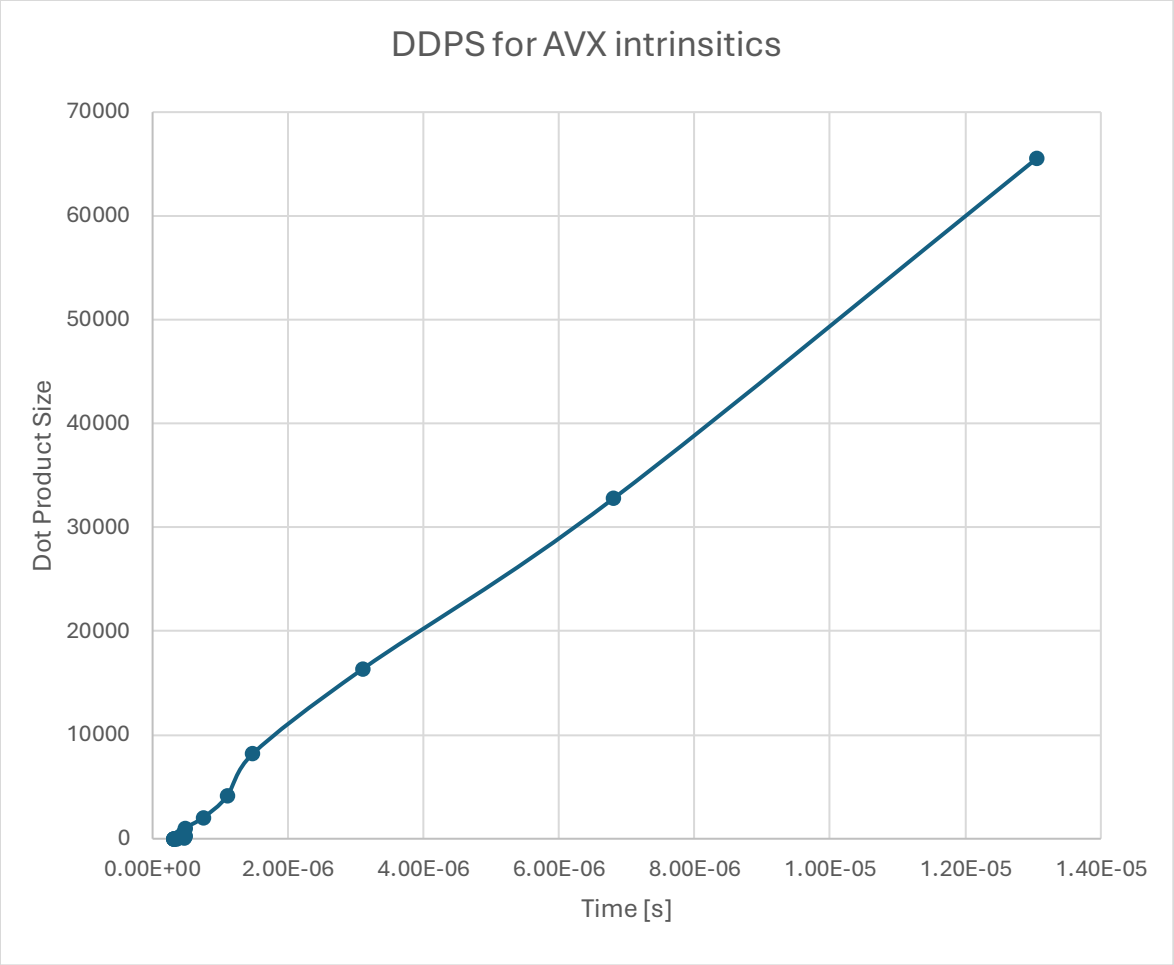


```cpp
#include <immintrin.h>
// sse AVX as it hass mm_dp_ps in it and sse 4.2 does not
#include <iostream>
#include "DotProductDPPS.h"
using namespace std;

float DotProductDPPS(const float* A, const float* B, int size) {
    __m256 sum = _mm256_setzero_ps();

    // We will process 8 elements at a time
    for (int i = 0; i < size; i += 8) {
        __m256 vec_a = _mm256_loadu_ps(&A[i]);
        __m256 vec_b = _mm256_loadu_ps(&B[i]);

        // Perform dot product on 8 elements
        __m256 dp = _mm256_dp_ps(vec_a, vec_b, 0xF1);
        // Control byte: 0xF1 for all elements becasue we set bottom as off and top on

        // Accumulate the result
        sum = _mm256_add_ps(sum, dp);
    }

    // Extract the horizontal sum of the final result
    float result[8];
    _mm256_storeu_ps(result, sum);

    // Accumulate the sum of all 8 floats in the result array
    float final_result = 0.0f;
    for (int i = 0; i < 8; ++i) {
        final_result += result[i];
    }

    return final_result;
}
```

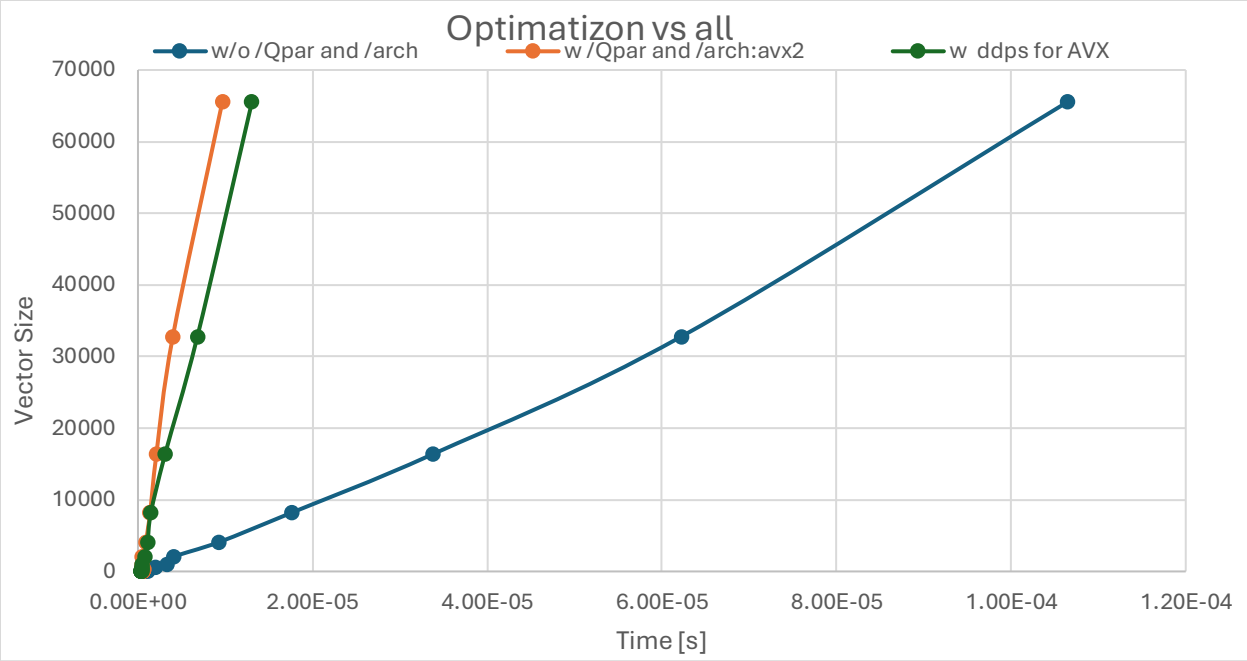
DotProductDPPS uses the Dpps for AVX intrinsics I had a problem for SSE:4.1

w ddps for AVX	
Time[s]	Dot Product Size
3.16E-07	1

3.57E-07	2
3.11E-07	4
3.11E-07	8
3.22E-07	16
3.64E-07	32
4.67E-07	64
4.00E-07	128
4.80E-07	256
4.60E-07	512
4.80E-07	1024
7.50E-07	2048
1.10E-06	4096
1.48E-06	8192
3.11E-06	16384
6.80E-06	32768
1.31E-05	65536



Task 5. Compare all plots in one figure.



Conclusion: We can clearly see a difference with using vector instruction from SMID compared to no optimizations.